Operating Systems - Synchronization Placement Quick Revision

Critical Section

Code segment accessing shared resources. Requirements: Mutual Exclusion, Progress, Bounded Waiting.

Race Condition

Occurs when multiple threads modify shared data simultaneously causing unpredictable results.

Mutual Exclusion

Only one thread/process enters critical section at a time.

Mutex

Lock with ownership. lock()->critical section->unlock(). Only owner unlocks. Best for protecting shared variables.

Semaphore

Counter-based synchronization. Binary (0/1) and Counting (0..N). wait()/signal() operations.

wait()

Acquire resource, decrement semaphore, block if unavailable.

signal()

Release resource, increment semaphore, wake one waiting thread.

Monitor

High-level synchronization with automatic locking/unlocking (e.g., Java synchronized).

Mutex vs Semaphore

Mutex: ownership, one resource. Semaphore: no ownership, one/many resources.

Why Monitors

Automatic synchronization, fewer programming errors.

Counting Semaphore Uses

Printer pool, DB connections, parking lot, thread pool.

Top Interview Q&A;

1. Race condition? Concurrent access causing incorrect results.
2. Critical section? Shared-resource code.
3. Mutex? Ownership lock.

4. Semaphore? Counter synchronization.
5. wait()? Decrement/block.
6. signal()? Increment/wake.
7. Monitor? Automatic mutual exclusion.
8. Mutex vs Semaphore? Ownership vs no ownership.
9. When counting semaphore? Multiple identical resources.
10. How mutex prevents race? Only one thread enters critical section.